

MAASHUKA

Premium Handloom Boutique Storefront — Production System Specification & Architectural Blueprint

Target Stack: Next.js 14/15 (App Router) | Supabase DB & Auth | Tailwind CSS | Sileo UI Toasts | Razorpay Engine

1. Executive Architecture Overview

The **Maashuka Customer Web Application** is engineered as a high-performance, secure, and visually striking headless e-commerce experience tailored for luxury retail. Leveraging a hybrid routing architecture powered by Next.js, data reads are primarily optimized using server-side rendering (SSR) and data pre-fetching streams, while highly interactive state interactions (such as the cart contextual actions, wishlist mutations, color-swatch mapping variables, and address autofill engines) are handled explicitly by decoupled client-side reactive frameworks.

The backend engine utilizes a highly relational **Supabase (PostgreSQL)** infrastructure with cascading referential integrity, structural indexing maps, and server-side row validation parameters to isolate administrative, logistical, and client execution bounds cleanly.

2. Complete Core Relational Database Schema

The storage system is built upon strict relational database management rules. Below are the precise table structures, foreign key constraints, data fields, and field schemas active across the Maashuka customer database ecosystem:

Table: categories

Column Name	Data Type	Constraints	Functional Description
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique internal tracking cluster node key.
name	VARCHAR(100)	NOT NULL	The user-visible catalog display label (e.g., "Banarasi Sarees").
slug	VARCHAR(100)	UNIQUE, NOT NULL	URL string safe indicator token utilized for category landing streams.
created_at	TIMESTAMPZ	DEFAULT NOW()	Audit record creation timestamp.

Table: **products**

COLUMN NAME	DATA TYPE	CONSTRAINTS	FUNCTIONAL DESCRIPTION
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique product row structural identifier.
category_id	UUID	REFERENCES categories(id) ON DELETE RESTRICT	Foreign key link connecting products to a parent category row segment.
name	VARCHAR(255)	NOT NULL	The visible product design name label.
slug	VARCHAR(255)	UNIQUE, NOT NULL	URL identifier token mapped by routing layers to detail specific views.
description	TEXT	-	Editorial copy catalog summary detailing weave metrics and fabrics.
base_price	NUMERIC(10,2)	NOT NULL, CHECK (base_price >= 0)	The standard baseline catalogue price parameter.
sale_price	NUMERIC(10,2)	CHECK (sale_price < base_price)	The modified active retail price when promotional rules apply.
on_sale	BOOLEAN	DEFAULT FALSE	Explicit structural switch parameter driving discount filtering mechanisms.
images	TEXT[]	DEFAULT '{}':TEXT[]	Array matrix containing URLs for general production gallery views.
created_at	TIMESTAMPTZ	DEFAULT NOW()	Creation audit index parameter.

Table: `product_variants`

COLUMN NAME	DATA TYPE	CONSTRAINTS	FUNCTIONAL DESCRIPTION
<code>id</code>	<code>UUID</code>	PRIMARY KEY, DEFAULT <code>gen_random_uuid()</code>	Unique stock keeping variant code node tracking identifier.
<code>product_id</code>	<code>UUID</code>	REFERENCES <code>products(id)</code> ON DELETE CASCADE	Parent structural reference linking directly back to general product definitions.
<code>sku</code>	<code>VARCHAR(100)</code>	UNIQUE, NOT NULL	Stock Keeping Unit string index variable for inventory systems.
<code>color</code>	<code>VARCHAR(50)</code>	NOT NULL	Text label descriptor mapped to active color profiles (e.g., "Crimson Rose").
<code>color_hex</code>	<code>VARCHAR(7)</code>	NOT NULL	Hexadecimal notation color value rendering visual circle controls.
<code>size</code>	<code>VARCHAR(20)</code>	NOT NULL	Sizing value parameter maps (e.g., "S", "M", "L", "XL", "Standard").
<code>variant_price</code>	<code>NUMERIC(10,2)</code>	NOT NULL	The exact current active retail pricing variable isolated for this row.
<code>pre_sale_price</code>	<code>NUMERIC(10,2)</code>	-	Backup repository parameter to restore standard baseline values post sales.
<code>stock_quantity</code>	<code>INT</code>	NOT NULL, DEFAULT 0, CHECK (<code>stock_quantity >= 0</code>)	Physical item balance threshold monitoring inventory availability bounds.
<code>variant_image</code>	<code>TEXT[]</code>	DEFAULT <code>'{}'::TEXT[]</code>	Specific localized imagery array showing specific color variants.

Table: **coupons**

COLUMN NAME	DATA TYPE	CONSTRAINTS	FUNCTIONAL DESCRIPTION
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique voucher validation row tracking identifier.
code	VARCHAR(50)	UNIQUE, NOT NULL	The token string format inserted by users (e.g., "MAASHUKA10").
discount_type	VARCHAR(20)	CHECK (discount_type IN ('percentage', 'fixed_amount'))	Mathematical operation paradigm driving price cuts.
discount_value	NUMERIC	NOT NULL, CHECK (discount_value > 0)	The raw absolute reduction value variable applied to invoices.
min_order_amount	NUMERIC	DEFAULT 0	Baseline cart subtotal required to allow voucher verification rules.
max_discount_amount	NUMERIC	-	Upper limits protection parameter to clamp percentage discount scaling.
is_active	BOOLEAN	DEFAULT TRUE	Master switch allowing or disallowing active operational lookups.
start_date	TIMESTAMPTZ	NOT NULL	The activation date-timestamp from which lookups succeed.
end_date	TIMESTAMPTZ	NOT NULL	The expiration stamp at which validation triggers automatically fail.
usage_limit	INT	-	Total number of times a coupon can be used globally.
used_count	INT	DEFAULT 0	Active aggregation variable track counting global vouchers finalized.

Table: coupon_usages

COLUMN NAME	DATA TYPE	CONSTRAINTS	FUNCTIONAL DESCRIPTION
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique user usage track node ledger variable index.
user_id	UUID	REFERENCES auth.users(id) ON DELETE CASCADE	Authenticated client profile row identifier mapping history records.
coupon_id	UUID	REFERENCES coupons(id) ON DELETE CASCADE	Target coupon descriptor identity linkage node parameter.
order_id	VARCHAR(100)	-	Linked finalized checkout voucher order identifier trace track.
used_at	TIMESTAMPTZ	DEFAULT NOW()	Final validation execution tracking timestamp parameter.

DATABASE INTEGRITY RULE: UNIQUE USER MULTI-USE GUARD

To ensure strict operational safety and prevent promotional coupon exploitation, a composite unique constraint is enforced across the usage tracker: `ALTER TABLE public.coupon_usages ADD CONSTRAINT unique_user_coupon UNIQUE (user_id, coupon_id);`

Table: orders

COLUMN NAME	DATA TYPE	CONSTRAINTS	FUNCTIONAL DESCRIPTION
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique store reference token tracking transaction records.
user_id	UUID	REFERENCES auth.users(id)	The ordering account identity node tracker configuration parameter.
subtotal_amount	NUMERIC(10,2)	NOT NULL	The initial cumulative price before any coupon deductions.
discount_amount	NUMERIC(10,2)	DEFAULT 0.00	The total value reduction applied via coupon validation metrics.
shipping_amount	NUMERIC(10,2)	DEFAULT 0.00	The delivery logistical freight fee parameter added to invoices.
payable_amount	NUMERIC(10,2)	NOT NULL	The net collection paid by the customer after processing discounts.
status	VARCHAR(50)	DEFAULT 'Pending'	Fulfillment monitoring state indicators ('Pending', 'Dispatched', 'Delivered').

3. Complete Step-by-Step Functional System Guides

This section describes the end-to-end execution paths of every user-visible and system-critical feature within the customer application layer.

3.1 Server-Side Data Fetching & On-Sale Listing Flow

The catalog listings are structured to utilize Next.js Server Components, fetching clean database arrays directly from the Supabase service layer without performing un-optimized API handshakes on the client viewport.

THE SERVER ARCHITECTURE PATH

1. The user routes to the `/sale` endpoint tracking node.
2. The Server Component initializes a secure Supabase connection module instance:

```
const supabase = await createClient();
```

3. An explicit query targets items where the `on_sale` parameters evaluate strictly to `true`:

```
const { data: prodData, error } = await supabase.from("products").select("*, categories(name, slug), product_variants(*)").eq("on_sale", true);
```

4. If error loops catch broken integrity paths, an intentional error exception triggers: `if (error) throw error;`
5. The structured server-side data payloads route cleanly downward into the Client Component view as rigid props:

```
<Sale Prod={prodData || []} Cat={catData || []} />
```

6. The client view receives these immutable arrays, tracking variables natively inside `useState` values, completely bypassing loading skeletons.

3.2 Premium Interactive Product Details REDESIGN Framework

The product details page is redesigned to optimize conversion pathways through deep layout scannability and asymmetric interactive grid alignments.

- **Asymmetric Responsive Grid Layout:** Formatted via a structural `lg:grid-cols-12` canvas. Image elements sit inside a flexible `lg:col-span-7` frame, while options selection blocks mount into a sticky `lg:col-span-5 lg:sticky lg:top-32` column segment ensuring the checkout call-to-action controls follow the user during scrolling.
- **Responsive Mobile Thumbnails track:** Thumbnails adjust layout based on screen width, transforming from an `overflow-x-auto` swipe row on mobile frames into a clean, rigid vertical column ribbon (`md:flex-col`) on desktop viewports.

- **Reactive Variant Selection Logic:** Color nodes map color arrays to round swatches, utilizing hexadecimal values (`color.hex`) for rendering. Clicking a swatch changes the color selection state, updates sizing grids, clears out-of-stock variations, and resets active image pointers seamlessly.
- **Dynamic Pricing Metric Matrix:** The active retail price displays dynamically based on user selections:

$$P_{\{current\}} = \text{If variant selected ? } V_{\{price\}} : \text{Base}_{\{price\}}$$

The savings percentage badge computes dynamically via a secure script layer expression:

$$\text{Discount} = \left\lfloor \frac{\text{Base}_{\{price\}} - P_{\{current\}}}{\text{Base}_{\{price\}}} \times 100 \right\rfloor$$

3.3 Intelligent Address Autofill & Dropdown Matching Engine

To eliminate customer typing errors and prevent shipping route failure penalties, the address checkout input fields combine public API lookups with an automated fallback mechanism.

LOGISTICS PINCODE FLOW PROTOCOL

1. The user inputs digits inside the address pincode structure container. A string sanitization rule filters out non-numeric characters: `e.target.value.replace(/\D/g, "")`.
2. When the clean input length reaches exactly 6 digits, the application handles a background dispatch request to the internal proxy endpoint: `fetch("/api/pincode")`.
3. **Low-Level Secure Server Proxy Handling:** Because public endpoints often suffer from expired certificate configurations (`CERT_HAS_EXPIRED`), the system drops down to a native core Node.js socket handshake layout, bypassing validation blocks safely:

```
const options = { hostname: "api.postalpincode.in", path: `/pincode/${pincode}`, method: "GET", rejectUnauthorized: false };
```

4. Client-Side Select Dropdown Mapping:

- If multiple post office distribution centers exist for the input code (e.g., 14 local sorting nodes found), the system saves the raw array inside the `postOffices` state vector.
- The UI adapts dynamically: the standard text input component changes into an explicit HTML `<select>` dropdown picker, allowing the client to select the correct specific area target.
- If the background API fails or runs into a timeout limit, the state array length evaluates to 0. The UI handles a fallback shift, keeping the standard text input visible so checkout flows never freeze.

3.4 Secure Coupon Code Verification & Usage Enforcement System

The coupon verification engine ensures code validations execute strictly behind protected backend server route checks before any order calculation logic modifies invoicing parameters.

```
[User Enters "MAASHUKA10"]
|
▼
[Next.js Server API Route /api/coupon/validate]
|
```

```

    ↳ Check 1: Is user logged in? (Supabase Auth Check) -> Fail: 401
    ↳ Check 2: Does code exist in DB? (Uppercase Match) -> Fail: 404
    ↳ Check 3: Is active & within Date boundaries? -> Fail: 400
    ↳ Check 4: Has global usage limit been reached? -> Fail: 400
    ↳ Check 5: Is cart subtotal >= min_order_amount? -> Fail: 400
    ↳ Check 6: Has user already used this coupon? -> Fail: 400 (Check coupon_usages)
    |
    ▼ [PASS ALL SWEEPS]
Compute Discount Value (Apply percentage clamp logic or fixed amount reductions)
    |
    ▼
[Return Validation Payload to UI & Finalize Order State Updates]

```

3.5 Enterprise Cookie Consent Popup Overlay Matrix

To satisfy privacy governance guidelines cleanly without generating interface block disruptions, the site runs an un-intrusive frosted glass contextual banner overlay window at the base tracking viewport tracking parameters.

- **Lifecycle Validation Trace:** Upon initial page load, a React `useEffect` checks browser state variables: `localStorage.getItem("maashuka_cookie_consent")`.
- **Controlled Entry Delay:** If no prior confirmation value is found, a 1500ms delay triggers before sliding the card animation cleanly into view (`z-[200] animate-in slide-in-from-bottom-10`). This delay prevents jarring shifting loops during core catalog asset rendering steps.
- **Interactive Preference Selection:** Clicking the "Accept" button saves an explicit flag: `localStorage.setItem("maashuka_cookie_consent", "true")`, which closes the layout and allows analytics scripts to execute smoothly.

4. Core Component Code Implementations & Layout Integration

This section lists the complete production code files implementing the global provider stacking layouts, navigation modifications, and relational tracking linkages:

4.1 Global Toast Provider Layer Separation (`components/ToastProvider.js`)

To ensure verification messages always display on top of absolute overlay modal frames (such as the address editing form window which rests at index `z-[150]`), the toaster provider is wrapped inside an elevated stacking context element:

```

"use client";
import { Toaster } from "sileo";

export default function ToastProvider() {
  return (
    <div className="relative z-[9999]">
      <Toaster position="bottom-center" />
    </div>
  );
}

```

4.2 High-Fidelity Menu Icon Links Integration

The client navigation menus are updated to use descriptive, context-aware vector indicators, ensuring a cohesive user profile menu interface.

```
// Saved Address Navigation Element Link Profile
import { MapPin } from "lucide-react";

<Link
  href="/address"
  onClick={() => setIsUserMenuOpen(false)}
  className="flex items-center gap-3 px-4 py-3 text-xs font-bold text-slate-600 hover:bg-
primary/5 hover:text-primary rounded-xl transition-colors"
>
  <MapPin size={16} strokeWidth={2} />
  <span>Saved Address</span>
</Link>

// Historical Order Records Lookup Element Link Profile
import { Package } from "lucide-react";

<Link
  href="/orders"
  onClick={() => setIsUserMenuOpen(false)}
  className="flex items-center gap-3 px-4 py-3 text-xs font-bold text-slate-600 hover:bg-
primary/5 hover:text-primary rounded-xl transition-colors"
>
  <Package size={16} strokeWidth={2} />
  <span>Orders Queue History</span>
</Link>
```

4.3 Secure Relational Parameter Layout Join Syntax

To map deep relational lookups within product component links without triggering runtime exceptions due to unsynchronized object keys, target links utilize proper parent chaining treatments with optional template fallback protections:

```
<Link
  key={`${item?.id || i}`}
  href={`\product/${item.product_variants?.product?.slug || ""}`}
  className="group block relative transition-all duration-300"
>
  {/* Component View Markup Node Data Display Layers */}
</Link>
```

5. Database Trigger Automation Setup (Supabase SQL)

To synchronize bulk pricing adjustments automatically across related tables, the database layer monitors modifications to product sale parameters and handles updating corresponding child variants instantly.

```

CREATE OR REPLACE FUNCTION public.sync_product_sale_prices()
RETURNS TRIGGER AS $$
BEGIN
    -- Evaluate if a product is toggled to sale status with a valid price
    IF NEW.on_sale = TRUE AND NEW.sale_price IS NOT NULL AND NEW.sale_price > 0 THEN
        UPDATE public.product_variants
        SET
            pre_sale_price = COALESCE(pre_sale_price, variant_price),
            variant_price = NEW.sale_price
        WHERE product_id = NEW.id;

    -- Revert variants to baseline catalogs when sale conditions clear
    ELIF (OLD.on_sale = TRUE AND NEW.on_sale = FALSE) OR (NEW.sale_price IS NULL) THEN
        UPDATE public.product_variants
        SET
            variant_price = COALESCE(pre_sale_price, variant_price),
            pre_sale_price = NULL
        WHERE product_id = NEW.id;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

-- Bind functional trigger script execution sequence to updates on specific tracking columns
CREATE OR REPLACE TRIGGER on_product_sale_change
AFTER UPDATE OF on_sale, sale_price ON public.products
FOR EACH ROW
EXECUTE FUNCTION public.sync_product_sale_prices();

```